

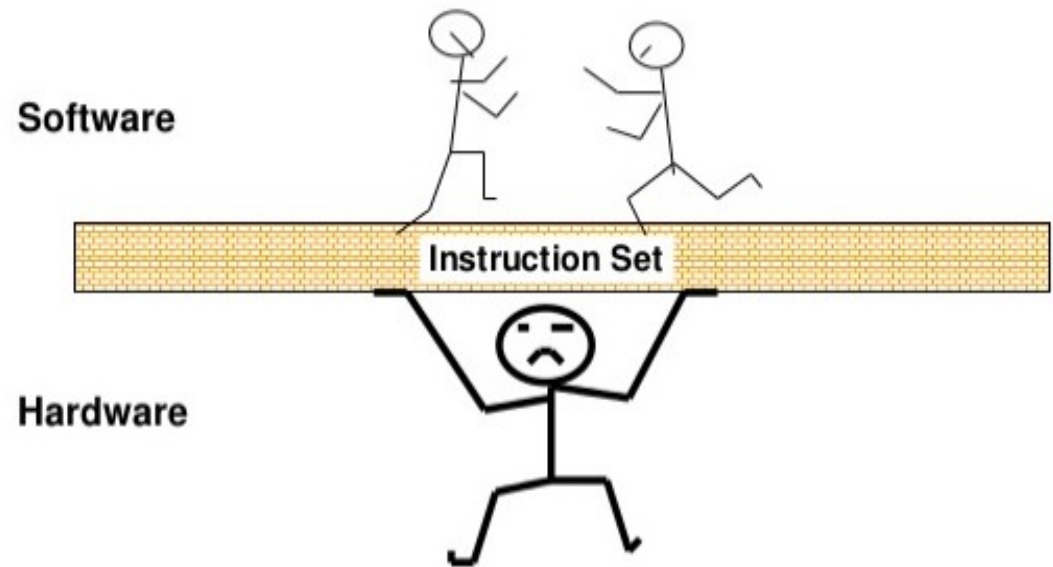
CS1CA Computer Systems Architecture
Instruction – machine language 3
Week 7 lecture 1 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher



(Figure: courtesy of the Internet)

In this lecture

- The following contents will be delivered
 - Operations in instruction sets
 - Encoding instruction sets
- The learning outcomes are listed below
 - To be aware of popular operations in instructions
 - To be able to describe the process of instruction encoding
 - To be able to compare and contrast the three encoding methods, i.e. variable, fixed length, and hybrid encoding
 - To be aware of desirable features in instruction set design

Operations in the instruction set (1)

Category of operators in instruction set architecture

Operator type	Examples
Arithmetic and logical	Integer arithmetic and logical operations: add, subtract, and, or, multiply, divide
Data transfer	Load-store, move ...
Control	Branch, jump, procedure call and return, traps
System	Operating system call, virtual memory...
Floating point	Floating point operations
Graphics	Pixel and vertex operations

Operations in the instruction set (2)

- **Rule:** the most widely executed instructions are the simple operations.

Intel 80×86 has top 10 simple operations: load, conditional branch, compare, store, add, and, sub, move register-register, call and return. These operations capture 96% of instructions.

- Instructions for control flow: three classes

Conditional branch: 75% (82% for floating-point)

Procedure call/return: 19% (8% for floating-point)

Jump: 6% (10% for floating-point)

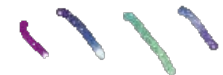
Note: % indicates control flow instructions possibly appear in a program.

Operations for media and DSPs

- Operations used in graphics are normally for single precision floating point and integer.
- *Single-instruction multiple-data (SIMD) or vector instructions are used to deal with narrow data in wider ALU (Arithmetic Logical Unit).*
- DSPs provide the following operations with different semantics for the real-time nature.
 - Arithmetic and logical
 - Data transfer
 - Control

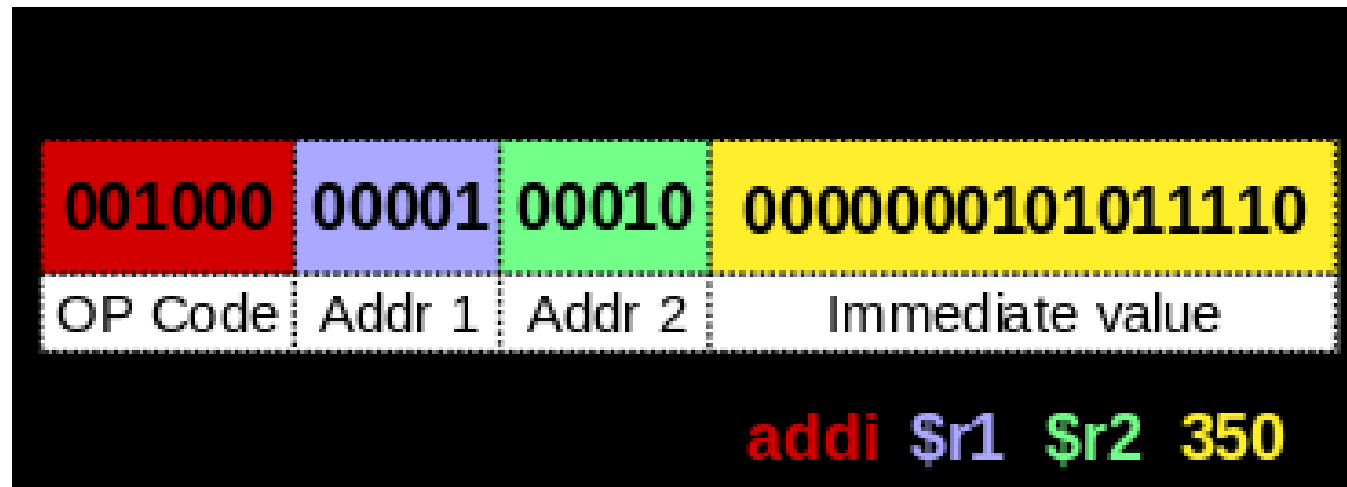
Add R10 (64 bit register)

R10



Add the first 4 bytes to the second 4 bytes and put the result back to the first 4 bytes.

Encoding an instruction set

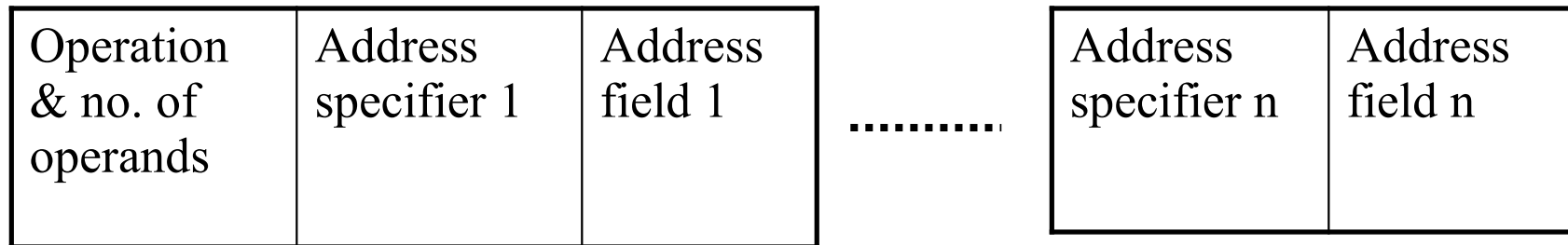


Encoding an instruction set

- Encoded instructions: a binary representation of operations, addressing modes, and operands for execution by the CPU.
- Number of registers and number of addressing modes affect instruction size. For example, Add R1, (R2)
- Ideally, from a programmer point of view, we prefer as many registers and addressing modes as possible.

Three basic types in instruction encoding (1)

- Variable length encoding: the number of bits varies for instructions; all addressing modes are feasible with all operations



e.g. Intel 80×86, VAX

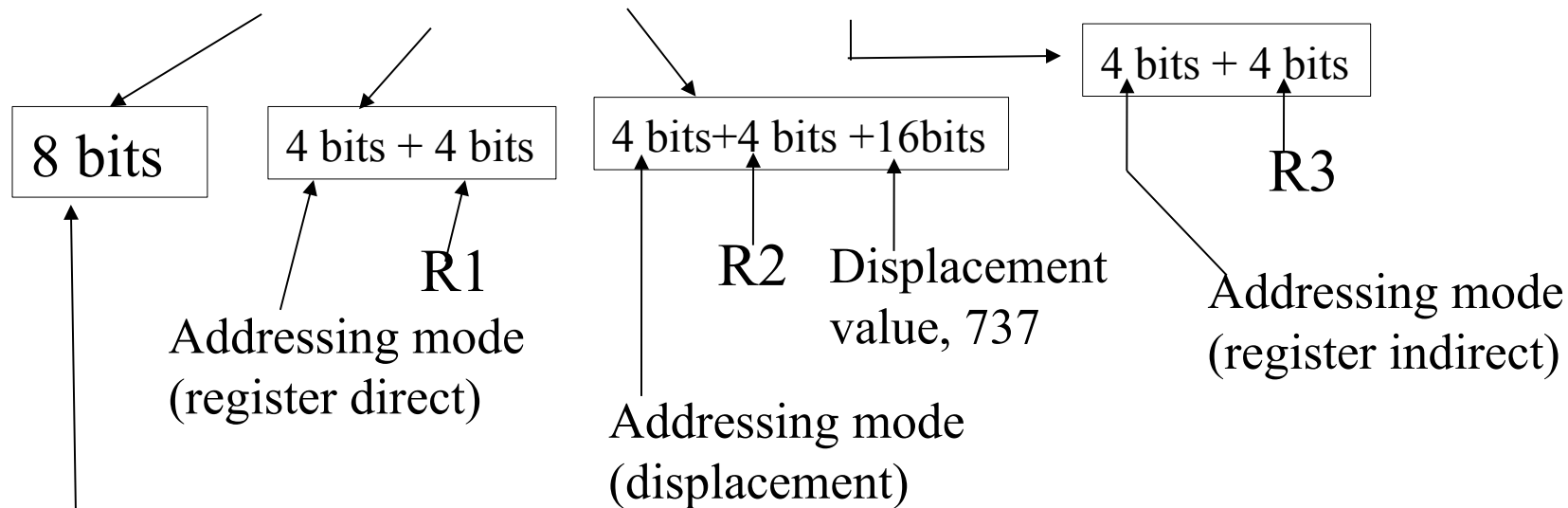
The length of Intel 80×86 instructions varies from 1-15 bytes.

Example: variable length encoding

- An instruction from VAX processor:

add13 R1, 737(R2), (R3)

Overall
 $1+1+3+1=6$
bytes in length



The opcode indicates that it is an addition operation with 32-bit integers, and no. of operands is 3.

Three basic types in instruction encoding (2)

- Fixed length encoding: all instructions have the same number of bits, e.g. 32 bits.
 - operation and addressing mode combined into opcode;
 - single size for all operations;
 - few addressing modes and few instructions

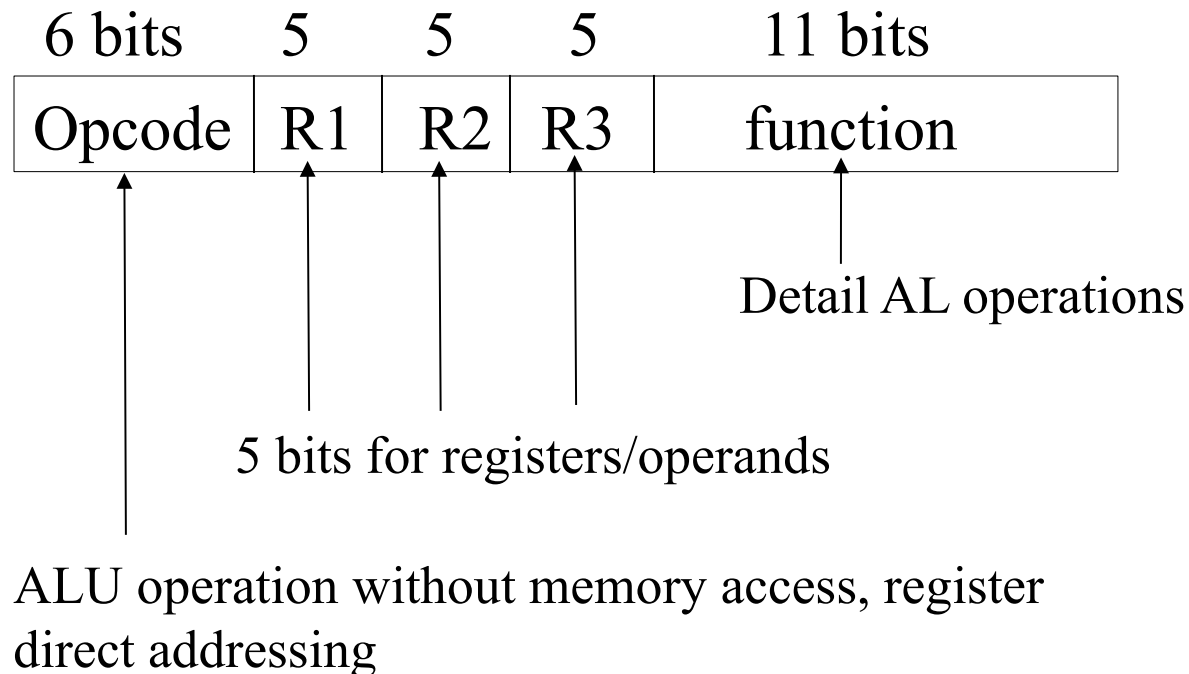
opcode	Address field 1	Address field 2	Address field 3
--------	--------------------	--------------------	--------------------

e.g. PowerPC, MIPS, DLX, ARM

Example: fixed length encoding

- An instruction from DLX processor:

Add R3, R1, R2



Overall, there are 32 bits (4 bytes) in length.

Three basic types in instruction encoding (3)

- Hybrid encoding: multiple instruction lengths, reduce variability and keep the flexibility

opcode	Address specifier	Address field
--------	-------------------	---------------

opcode	Address specifier 1	Address specifier 2	Address field
--------	---------------------	---------------------	---------------

e.g. IBM 360/370, MIPS16, ...

For example,
Add16 R3, R1, R2

Fixed vs variable length encoding

(refer to register-register and register-memory architecture)

- Fixed Length
 - Simple, easy for decoding
 - Larger code size (higher instruction count)
- Variable Length
 - More complex, harder to decode
 - More compact, efficient use of memory
 - Complex pipeline: instructions vary greatly in both size and amount of work to be performed (we shall discuss this when we come to the topic of “pipelining”)

Proposed architecture: desirable features

- General purpose registers
- Load-store (register-register) architecture
- Few addressing modes
- Simple instructions
- Data size: byte multiplied
- Hybrid encoding: fixed instruction encoding for performance, and variable instruction encoding for code size and flexibility
- Reduced instruction set.

End of the lecture

Questions and exercises

- Operations in instruction sets
 - What are popular operations involved in instructions?
 - For media and DSP, what do we need to take into account in designing instruction sets?
- Encoding instructions
 - What is the process of encoding instructions?
 - What types of encoding can we use?
 - Compare and contrast fixed length and variable length encoding
 - Using examples, explain how to implement addressing modes in encoding instructions?